

lunarlander_rl

March 23, 2026

1 LunarLander: Comparación de Algoritmos de RL

1.1 Objetivos de este Notebook

1. Comparar PPO vs DQN vs A2C: ¿Cuál funciona mejor?
2. Entender qué significa “resolver” un entorno
3. Estudiar el efecto de gamma (horizonte temporal)
4. Experimentar con learning rate
5. Probar versión continua con SAC

1.2 Prerequisitos

```
pip install stable-baselines3 gymnasium[box2d]
```

```
[1]: !pip install stable-baselines3 "stable-baselines3[extra]" "gymnasium[box2d]"
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: stable-baselines3 in
/home/jeironpro/.local/lib/python3.14/site-packages (2.7.1)
Requirement already satisfied: gymnasium[box2d] in
/home/jeironpro/.local/lib/python3.14/site-packages (1.2.2)
Requirement already satisfied: numpy<3.0,>=1.20 in /usr/lib64/python3.14/site-
packages (from stable-baselines3) (2.3.5)
Requirement already satisfied: torch<3.0,>=2.3 in
/home/jeironpro/.local/lib/python3.14/site-packages (from stable-baselines3)
(2.10.0)
Requirement already satisfied: cloudpickle in
/home/jeironpro/.local/lib/python3.14/site-packages (from stable-baselines3)
(3.1.2)
Requirement already satisfied: pandas in
/home/jeironpro/.local/lib/python3.14/site-packages (from stable-baselines3)
(3.0.1)
Requirement already satisfied: matplotlib in
/home/jeironpro/.local/lib/python3.14/site-packages (from stable-baselines3)
(3.10.8)
Requirement already satisfied: typing-extensions>=4.3.0 in
/usr/lib/python3.14/site-packages (from gymnasium[box2d]) (4.15.0)
Requirement already satisfied: farama-notifications>=0.0.1 in
```

/home/jeironpro/.local/lib/python3.14/site-packages (from gymnasium[box2d])
 (0.0.4)
 Requirement already satisfied: filelock in /usr/lib/python3.14/site-packages
 (from torch<3.0,>=2.3->stable-baselines3) (3.15.4)
 Requirement already satisfied: setuptools in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (80.10.2)
 Requirement already satisfied: sympy>=1.13.3 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (1.14.0)
 Requirement already satisfied: networkx>=2.5.1 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (3.6.1)
 Requirement already satisfied: jinja2 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (3.1.6)
 Requirement already satisfied: fsspec>=0.8.5 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (2026.2.0)
 Requirement already satisfied: cuda-bindings==12.9.4 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (12.9.4)
 Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (12.8.93)
 Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (12.8.90)
 Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (12.8.90)
 Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (9.10.2.21)
 Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (12.8.4.1)
 Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (11.3.3.83)
 Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (10.3.9.90)
 Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from
 torch<3.0,>=2.3->stable-baselines3) (11.7.3.90)
 Requirement already satisfied: nvidia-cuspars-cu12==12.5.8.93 in
 /home/jeironpro/.local/lib/python3.14/site-packages (from

```

torch<3.0,>=2.3->stable-baselines3) (12.5.8.93)
Requirement already satisfied: nvidia-cusparse-cu12==0.7.1 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (3.4.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (1.13.1.3)
Requirement already satisfied: triton==3.6.0 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
torch<3.0,>=2.3->stable-baselines3) (3.6.0)
Requirement already satisfied: cuda-pathfinder~=1.1 in
/home/jeironpro/.local/lib/python3.14/site-packages (from cuda-
bindings==12.9.4->torch<3.0,>=2.3->stable-baselines3) (1.4.3)
Collecting opencv-python (from stable-baselines3[extra])
  Using cached
opencv_python-4.13.0.92-cp37-abi3-manylinux_2_28_x86_64.whl.metadata (19 kB)
Requirement already satisfied: pygame in
/home/jeironpro/.local/lib/python3.14/site-packages (from stable-
baselines3[extra]) (2.6.1)
Requirement already satisfied: tensorboard>=2.9.1 in
/home/jeironpro/.local/lib/python3.14/site-packages (from stable-
baselines3[extra]) (2.20.0)
Requirement already satisfied: psutil in /usr/lib64/python3.14/site-packages
(from stable-baselines3[extra]) (7.0.0)
Collecting tqdm (from stable-baselines3[extra])
  Using cached tqdm-4.67.3-py3-none-any.whl.metadata (57 kB)
Collecting rich (from stable-baselines3[extra])
  Using cached rich-14.3.3-py3-none-any.whl.metadata (18 kB)
Collecting ale-py>=0.9.0 (from stable-baselines3[extra])
  Using cached ale_py-0.11.2.tar.gz (512 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Installing backend dependencies ... done
  Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: pillow in /usr/lib64/python3.14/site-
packages (from stable-baselines3[extra]) (11.3.0)

```

Requirement already satisfied: box2d-py==2.3.5 in
/home/jeironpro/.local/lib/python3.14/site-packages (from gymnasium[box2d])
(2.3.5)

Requirement already satisfied: swig==4.* in
/home/jeironpro/.local/lib/python3.14/site-packages (from gymnasium[box2d])
(4.4.1)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
sympy>=1.13.3->torch<3.0,>=2.3->stable-baselines3) (1.3.0)

Requirement already satisfied: absl-py>=0.4 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
tensorboard>=2.9.1->stable-baselines3[extra]) (2.4.0)

Requirement already satisfied: grpcio>=1.48.2 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
tensorboard>=2.9.1->stable-baselines3[extra]) (1.78.0)

Requirement already satisfied: markdown>=2.6.8 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
tensorboard>=2.9.1->stable-baselines3[extra]) (3.10.2)

Requirement already satisfied: packaging in
/home/jeironpro/.local/lib/python3.14/site-packages (from
tensorboard>=2.9.1->stable-baselines3[extra]) (26.0)

Requirement already satisfied: protobuf!=4.24.0,>=3.19.6 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
tensorboard>=2.9.1->stable-baselines3[extra]) (7.34.1)

Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
tensorboard>=2.9.1->stable-baselines3[extra]) (0.7.2)

Requirement already satisfied: werkzeug>=1.0.1 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
tensorboard>=2.9.1->stable-baselines3[extra]) (3.1.6)

Requirement already satisfied: markupsafe>=2.1.1 in
/home/jeironpro/.local/lib/python3.14/site-packages (from
werkzeug>=1.0.1->tensorboard>=2.9.1->stable-baselines3[extra]) (3.0.3)

Requirement already satisfied: contourpy>=1.0.1 in
/home/jeironpro/.local/lib/python3.14/site-packages (from matplotlib->stable-
baselines3) (1.3.3)

Requirement already satisfied: cycler>=0.10 in
/home/jeironpro/.local/lib/python3.14/site-packages (from matplotlib->stable-
baselines3) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in
/home/jeironpro/.local/lib/python3.14/site-packages (from matplotlib->stable-
baselines3) (4.62.1)

Requirement already satisfied: kiwisolver>=1.3.1 in
/home/jeironpro/.local/lib/python3.14/site-packages (from matplotlib->stable-
baselines3) (1.5.0)

Requirement already satisfied: pyparsing>=3 in /usr/lib/python3.14/site-packages
(from matplotlib->stable-baselines3) (3.1.2)

Requirement already satisfied: python-dateutil>=2.7 in /usr/lib/python3.14/site-

```

packages (from matplotlib->stable-baselines3) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/lib/python3.14/site-packages
(from python-dateutil>=2.7->matplotlib->stable-baselines3) (1.17.0)
Collecting markdown-it-py>=2.2.0 (from rich->stable-baselines3[extra])
  Using cached markdown_it_py-4.0.0-py3-none-any.whl.metadata (7.3 kB)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/home/jeironpro/.local/lib/python3.14/site-packages (from rich->stable-
baselines3[extra]) (2.19.2)
Collecting mdurl~=0.1 (from markdown-it-py>=2.2.0->rich->stable-
baselines3[extra])
  Using cached mdurl-0.1.2-py3-none-any.whl.metadata (1.6 kB)
Using cached opencv_python-4.13.0.92-cp37-abi3-manylinux_2_28_x86_64.whl (72.9
MB)
Using cached rich-14.3.3-py3-none-any.whl (310 kB)
Using cached markdown_it_py-4.0.0-py3-none-any.whl (87 kB)
Using cached mdurl-0.1.2-py3-none-any.whl (10.0 kB)
Using cached tqdm-4.67.3-py3-none-any.whl (78 kB)
Building wheels for collected packages: ale-py
  Building wheel for ale-py (pyproject.toml) ... done
  Created wheel for ale-py:
filename=ale_py-0.11.2-cp314-cp314-linux_x86_64.whl size=1047344
sha256=c809943be5fcab848ce05708f3f6575f1a0ee424ba19827e0ea5281483c7c970
  Stored in directory: /home/jeironpro/.cache/pip/wheels/59/d5/74/4c0284b172c3ea
539024945a3223bab57b7673d15208c254d8
Successfully built ale-py
Installing collected packages: tqdm, opencv-python, mdurl, ale-py, markdown-it-
py, rich

```

6/6

```

[rich][32m5/6 [rich]v-python]
Successfully installed ale-py-0.11.2 markdown-it-py-4.0.0 mdurl-0.1.2
opencv-python-4.13.0.92 rich-14.3.3 tqdm-4.67.3

```

```

[2]: import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict
import time

# Gymnasium
import gymnasium as gym

# Stable-Baselines3
from stable_baselines3 import PPO, DQN, A2C, SAC
from stable_baselines3.common.evaluation import evaluate_policy
from stable_baselines3.common.callbacks import BaseCallback
from stable_baselines3.common.monitor import Monitor

# Verificar Box2D

```

```

try:
    env_test = gym.make("LunarLander-v3")
    env_test.close()
    LUNAR_AVAILABLE = True
    print("LunarLander disponible")
except:
    LUNAR_AVAILABLE = False
    print("Instalar con: pip install gymnasium[box2d]")

print(f"Gymnasium disponible")

```

LunarLander disponible

Gymnasium disponible

<frozen importlib._bootstrap>:491: RuntimeWarning: Your system is avx2 capable but pygame was not built with support for it. The performance of some of your blits could be adversely affected. Consider enabling compile time detection with environment variables like PYGAME_DETECT_AVX2=1 if you are compiling without cross compilation.

/home/jeironpro/.local/lib/python3.14/site-packages/pygame/pkgdata.py:25:

UserWarning: pkg_resources is deprecated as an API. See

https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.

```
from pkg_resources import resource_stream, resource_exists
```

2 1. Descripción del Entorno

2.1 LunarLander-v3

Aterrizar un módulo lunar de forma segura:

```

      ← Módulo lunar
    / | \
   /  |  \ ← Propulsores

  /          \
 /            \ ← Zona de aterrizaje

```

2.1.1 Criterio de “Resolver”

El entorno se considera **resuelto** cuando el agente obtiene una recompensa promedio de +200 o más.

```

[3]: if LUNAR_AVAILABLE:
      env = gym.make("LunarLander-v3")

```

```

print("="*60)
print("ENTORNO: LunarLander-v3")
print("="*60)
print(f"\nEspacio de observación: {env.observation_space}")
print(f" - 8 dimensiones continuas")
print(f"\nEspacio de acciones: {env.action_space}")
print(f" - 4 acciones discretas")

# Detalles
print("\n" + "-"*60)
print("OBSERVACIÓN (8D):")
print("-"*60)
obs_desc = [
    "Posición X",
    "Posición Y",
    "Velocidad X",
    "Velocidad Y",
    "Ángulo",
    "Velocidad angular",
    "Pierna izquierda en contacto (0/1)",
    "Pierna derecha en contacto (0/1)"
]
for i, desc in enumerate(obs_desc):
    print(f" [{i}] {desc}")

print("\n" + "-"*60)
print("ACCIONES (4):")
print("-"*60)
acciones = [
    "No hacer nada",
    "Motor izquierdo",
    "Motor principal (abajo)",
    "Motor derecho"
]
for i, desc in enumerate(acciones):
    print(f" [{i}] {desc}")

print("\n" + "-"*60)
print("RECOMPENSAS:")
print("-"*60)
print(" Aterrizar en zona: +100 a +140")
print(" Cada pierna en contacto: +10")
print(" Motor principal encendido: -0.3/frame")
print(" Motor lateral encendido: -0.03/frame")
print(" Crash: -100")
print(" Salir de pantalla: -100")
print("\n RESUELTO: recompensa promedio >= 200")

```

```
env.close()
```

```
=====
ENTORNO: LunarLander-v3
=====
```

```
Espacio de observación: Box([ -2.5      -2.5      -10.      -10.
-6.2831855 -10.
    -0.      -0.      ], [ 2.5      2.5      10.      10.
6.2831855 10.
    1.      1.      ], (8,)), float32)
- 8 dimensiones continuas
```

```
Espacio de acciones: Discrete(4)
- 4 acciones discretas
```

```
-----
OBSERVACIÓN (8D):
-----
```

```
[0] Posición X
[1] Posición Y
[2] Velocidad X
[3] Velocidad Y
[4] Ángulo
[5] Velocidad angular
[6] Pierna izquierda en contacto (0/1)
[7] Pierna derecha en contacto (0/1)
```

```
-----
ACCIONES (4):
-----
```

```
[0] No hacer nada
[1] Motor izquierdo
[2] Motor principal (abajo)
[3] Motor derecho
```

```
-----
RECOMPENSAS:
-----
```

```
Aterrizar en zona: +100 a +140
Cada pierna en contacto: +10
Motor principal encendido: -0.3/frame
Motor lateral encendido: -0.03/frame
Crash: -100
Salir de pantalla: -100
```

```
RESUELTO: recompensa promedio >= 200
```

3 2. Análisis de Algoritmos

3.1 PPO vs DQN vs A2C

Aspecto	PPO	DQN	A2C
Tipo	Policy Gradient	Value-based	Policy Gradient
On/Off-policy	On-policy	Off-policy	On-policy
Replay Buffer	No	Sí	No
Sample Efficiency	Media	Alta	Baja
Estabilidad	Alta	Media	Media
Acciones Continuas	Sí	No	Sí

3.1.1 ¿Cuándo usar cada uno?

- **PPO**: Algoritmo por defecto, robusto y estable
 - **DQN**: Cuando tienes acciones discretas y quieres sample efficiency
 - **A2C**: Similar a PPO pero más simple, útil para debugging
-

4 3. Código Base

```
[4]: class RewardCallback(BaseCallback):
    """Callback para registrar recompensas."""

    def __init__(self, verbose=0):
        super().__init__(verbose)
        self.episode_rewards = []
        self.episode_lengths = []

    def _on_step(self) -> bool:
        for info in self.locals.get('infos', []):
            if 'episode' in info:
                self.episode_rewards.append(info['episode']['r'])
                self.episode_lengths.append(info['episode']['l'])
        return True

def entrenar(algoritmo, timesteps=50000, **kwargs):
    """
    Entrena un algoritmo en LunarLander.

    Args:
        algoritmo: "PPO", "DQN", "A2C" o "SAC"
        timesteps: Pasos de entrenamiento
```

```

        kwargs: Hiperparámetros adicionales
        """
    env = gym.make("LunarLander-v3")
    env = Monitor(env)

    # Configuración por defecto
    config = {
        "PPO": {"learning_rate": 0.0003, "n_steps": 2048, "batch_size": 64,
        ↪ "gamma": 0.99},
        "DQN": {"learning_rate": 0.0001, "buffer_size": 100000, "gamma": 0.99},
        "A2C": {"learning_rate": 0.0007, "n_steps": 5, "gamma": 0.99},
    }

    params = config.get(algoritmo, {})
    params.update(kwargs)

    # Crear modelo
    if algoritmo == "PPO":
        model = PPO("MlpPolicy", env, verbose=0, **params)
    elif algoritmo == "DQN":
        model = DQN("MlpPolicy", env, verbose=0, **params)
    elif algoritmo == "A2C":
        model = A2C("MlpPolicy", env, verbose=0, **params)
    else:
        raise ValueError(f"Algoritmo no soportado: {algoritmo}")

    callback = RewardCallback()
    model.learn(total_timesteps=timesteps, callback=callback, progress_bar=True)

    env.close()
    return model, callback

print("Funciones de entrenamiento cargadas")

```

Funciones de entrenamiento cargadas

5 4. VARIANTE A: Comparación PPO vs DQN vs A2C

```

[5]: def comparar_algoritmos(timesteps=50000):
    """
    Compara PPO, DQN y A2C en LunarLander.
    """
    if not LUNAR_AVAILABLE:
        print("LunarLander no disponible")
        return {}

```

```

print("="*60)
print("VARIANTE A: Comparación PPO vs DQN vs A2C")
print("="*60)

resultados = {}

for algo in ["PPO", "DQN", "A2C"]:
    print(f"\nEntrenando {algo}...")
    t0 = time.time()
    model, callback = entrenar(algo, timesteps)
    tiempo = time.time() - t0

    # Evaluar
    env_eval = gym.make("LunarLander-v3")
    mean_reward, std_reward = evaluate_policy(model, env_eval,
↪n_eval_episodes=10)
    env_eval.close()

    resultados[algo] = {
        'rewards': callback.episode_rewards,
        'mean': mean_reward,
        'std': std_reward,
        'time': tiempo,
        'solved': mean_reward >= 200
    }

    status = "RESUELTO" if mean_reward >= 200 else "No resuelto"
    print(f"  {algo}: {mean_reward:.1f} ± {std_reward:.1f} ({status})")

return resultados

if LUNAR_AVAILABLE:
    resultados_a = comparar_algoritmos(timesteps=30000)

```

```

=====
VARIANTE A: Comparación PPO vs DQN vs A2C
=====

```

Entrenando PPO...

```

/home/jeironpro/.local/lib/python3.14/site-packages/rich/live.py:260:
↪UserWarning: install "ipywidgets" for Jupyter
support
warnings.warn('install "ipywidgets" for Jupyter support')

```

```
/home/jeironpro/.local/lib/python3.14/site-  
packages/stable_baselines3/common/evaluation.py:70: UserWarning: Evaluation  
environment is not wrapped with a ``Monitor`` wrapper. This may result in  
reporting modified episode lengths and rewards, if other wrappers happen to  
modify these. Consider wrapping environment first with ``Monitor`` wrapper.
```

```
warnings.warn(  

```

```
PPO: -3170.3 ± 2776.4 (No resuelto)
```

Entrenando DQN...

```
DQN: -232.5 ± 62.4 (No resuelto)
```

Entrenando A2C...

```
A2C: -21.2 ± 95.3 (No resuelto)
```

```
[6]: # Visualizar comparación  
if LUNAR_AVAILABLE and resultados_a:  
    fig, axes = plt.subplots(1, 3, figsize=(15, 4))  
  
    # Curvas de aprendizaje  
    for algo, data in resultados_a.items():  
        rewards = data['rewards']  
        if len(rewards) > 20:  
            smoothed = np.convolve(rewards, np.ones(20)/20, mode='valid')  
            axes[0].plot(smoothed, label=algo)  
    axes[0].axhline(y=200, color='g', linestyle='--', label='Resuelto (200)')  
    axes[0].set_xlabel('Episodio')  
    axes[0].set_ylabel('Recompensa')  
    axes[0].set_title('Curvas de Aprendizaje')  
    axes[0].legend()  
    axes[0].grid(True, alpha=0.3)  
  
    # Recompensa final  
    algos = list(resultados_a.keys())  
    means = [resultados_a[a]['mean'] for a in algos]  
    stds = [resultados_a[a]['std'] for a in algos]  
    colors = ['green' if resultados_a[a]['solved'] else 'red' for a in algos]  
    axes[1].bar(algos, means, yerr=stds, capsize=5, color=colors, alpha=0.7)  
    axes[1].axhline(y=200, color='g', linestyle='--')  
    axes[1].set_ylabel('Recompensa Media')  
    axes[1].set_title('Rendimiento Final')  
  
    # Tiempo de entrenamiento  
    times = [resultados_a[a]['time'] for a in algos]
```

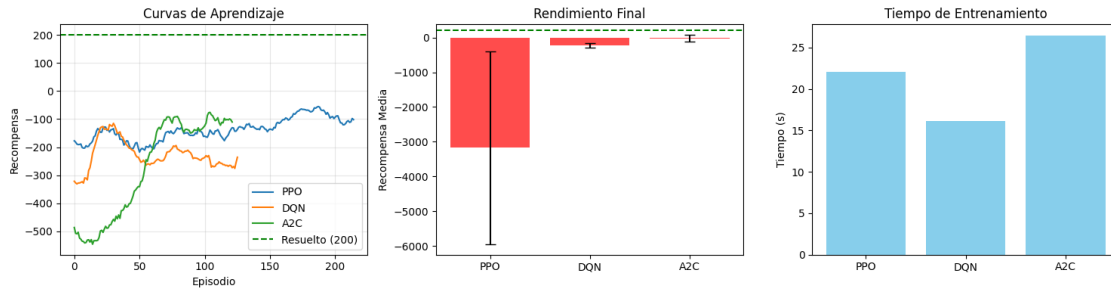
```

axes[2].bar(algos, times, color='skyblue')
axes[2].set_ylabel('Tiempo (s)')
axes[2].set_title('Tiempo de Entrenamiento')

plt.tight_layout()
plt.show()

# Tabla resumen
print("\n" + "="*60)
print("RESUMEN")
print("="*60)
print(f"\n{'Algoritmo':<10} {'Recompensa':<20} {'Tiempo (s)':<12}{'Estado':<12}")
print("-" * 55)
for algo in algos:
    d = resultados_a[algo]
    status = "RESUELTO" if d['solved'] else "No resuelto"
    print(f"{algo:<10} {d['mean']:.1f} ± {d['std']:.1f}{'':8} {d['time']:<12.1f} {status}")

```



=====

RESUMEN

=====

Algoritmo	Recompensa	Tiempo (s)	Estado
PPO	-3170.3 ± 2776.4	22.1	No resuelto
DQN	-232.5 ± 62.4	16.1	No resuelto
A2C	-21.2 ± 95.3	26.5	No resuelto

6 5. VARIANTE B: LunarLander Continuo con SAC

6.0.1 Diferencia: Acciones Continuas

Discreto	Continuo
4 acciones (0,1,2,3)	2 valores continuos [-1,1]
Motor encendido/apagado	Control fino de potencia

```
[7]: def entrenar_continuo(timesteps=50000):
    """
    Entrena SAC en LunarLanderContinuous.
    """
    print("="*60)
    print("VARIANTE B: LunarLander Continuo con SAC")
    print("="*60)

    try:
        env = gym.make("LunarLanderContinuous-v3")
        env = Monitor(env)

        print(f"\nEspacio de acciones: {env.action_space}")
        print(" [0]: Motor principal (-1 a 1)")
        print(" [1]: Motor lateral (-1 a 1)")

        print("\nEntrenando SAC...")

        model = SAC(
            "MlpPolicy", env, verbose=0,
            learning_rate=0.0003,
            buffer_size=100000,
            batch_size=256,
            gamma=0.99,
            tau=0.005
        )

        callback = RewardCallback()
        model.learn(total_timesteps=timesteps, callback=callback,
        ↪ progress_bar=True)

        # Evaluar
        env_eval = gym.make("LunarLanderContinuous-v3")
        mean_reward, std_reward = evaluate_policy(model, env_eval,
        ↪ n_eval_episodes=10)
        env_eval.close()

        status = "RESUELTO" if mean_reward >= 200 else "No resuelto"
        print(f"\nResultado SAC: {mean_reward:.1f} ± {std_reward:.1f}")
        ↪ ({status})

        env.close()
```

```

        return model, callback

    except Exception as e:
        print(f"Error: {e}")
        return None, None

if LUNAR_AVAILABLE:
    model_sac, cb_sac = entrenar_continuo(timesteps=30000)

```

=====

VARIANTE B: LunarLander Continuo con SAC

=====

Espacio de acciones: Box(-1.0, 1.0, (2,)), float32)

[0]: Motor principal (-1 a 1)

[1]: Motor lateral (-1 a 1)

Entrenando SAC...

Resultado SAC: 5.3 ± 144.7 (No resuelto)

7 6. VARIANTE C: Estudio de Gamma (Horizonte Temporal)

7.0.1 ¿Qué es gamma ()?

El factor de descuento determina cuánto valora el agente las recompensas futuras:

- = **0.9**: Horizonte corto, prefiere recompensas inmediatas
- = **0.99**: Horizonte largo, planifica a futuro
- = **0.999**: Horizonte muy largo

Return = $r + \gamma r + \gamma^2 r + \gamma^3 r + \dots$

```

[8]: def estudiar_gamma(timesteps=30000):
    """
    Estudia el efecto de diferentes valores de gamma.
    """
    if not LUNAR_AVAILABLE:
        return {}

    print("="*60)
    print("VARIANTE C: Estudio de Gamma")
    print("="*60)

    gammas = [0.9, 0.95, 0.99, 0.999]

```

```

resultados = {}

for gamma in gammas:
    print(f"\nEntrenando PPO con gamma={gamma}...")
    model, callback = entrenar("PPO", timesteps, gamma=gamma)

    env_eval = gym.make("LunarLander-v3")
    mean_reward, std_reward = evaluate_policy(model, env_eval,
    ↪n_eval_episodes=10)
    env_eval.close()

    resultados[gamma] = {
        'rewards': callback.episode_rewards,
        'mean': mean_reward,
        'std': std_reward
    }
    print(f"  gamma={gamma}: {mean_reward:.1f} ± {std_reward:.1f}")

return resultados

if LUNAR_AVAILABLE:
    resultados_c = estudiar_gamma(timesteps=20000)

```

```

=====
VARIANTE C: Estudio de Gamma
=====

```

Entrenando PPO con gamma=0.9...

gamma=0.9: -803.2 ± 169.5

Entrenando PPO con gamma=0.95...

gamma=0.95: -666.7 ± 117.9

Entrenando PPO con gamma=0.99...

gamma=0.99: -1143.2 ± 633.8

Entrenando PPO con gamma=0.999...

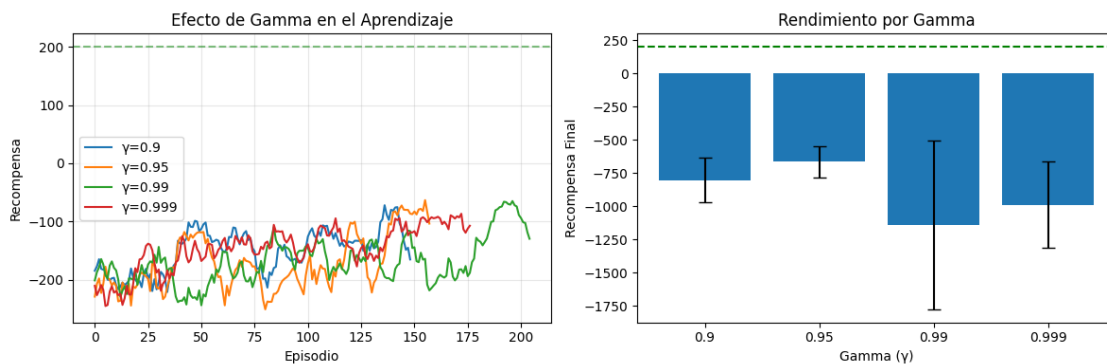
gamma=0.999: -989.2 ± 326.0


```
[9]: # Visualizar efecto de gamma
if LUNAR_AVAILABLE and 'resultados_c' in dir() and resultados_c:
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))

    # Curvas
    for gamma, data in resultados_c.items():
        rewards = data['rewards']
        if len(rewards) > 10:
            smoothed = np.convolve(rewards, np.ones(10)/10, mode='valid')
            axes[0].plot(smoothed, label=f'{gamma}')
    axes[0].axhline(y=200, color='g', linestyle='--', alpha=0.5)
    axes[0].set_xlabel('Episodio')
    axes[0].set_ylabel('Recompensa')
    axes[0].set_title('Efecto de Gamma en el Aprendizaje')
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

    # Barras
    gammas = list(resultados_c.keys())
    means = [resultados_c[g]['mean'] for g in gammas]
    stds = [resultados_c[g]['std'] for g in gammas]
    axes[1].bar([str(g) for g in gammas], means, yerr=stds, capsize=5)
    axes[1].axhline(y=200, color='g', linestyle='--')
    axes[1].set_xlabel('Gamma (γ)')
    axes[1].set_ylabel('Recompensa Final')
    axes[1].set_title('Rendimiento por Gamma')

plt.tight_layout()
plt.show()
```



8 7. VARIANTE D: Learning Rate Sweep

```
[10]: def estudiar_learning_rate(timesteps=30000):
    """
    Estudia el efecto del learning rate.
    """
    if not LUNAR_AVAILABLE:
        return {}

    print("="*60)
    print("VARIANTE D: Learning Rate Sweep")
    print("="*60)

    lrs = [1e-4, 3e-4, 1e-3, 3e-3]
    resultados = {}

    for lr in lrs:
        print(f"\nEntrenando PPO con lr={lr}...")
        model, callback = entrenar("PPO", timesteps, learning_rate=lr)

        env_eval = gym.make("LunarLander-v3")
        mean_reward, std_reward = evaluate_policy(model, env_eval,
↪n_eval_episodes=10)
        env_eval.close()

        resultados[lr] = {
            'rewards': callback.episode_rewards,
            'mean': mean_reward,
            'std': std_reward
        }
        print(f"  lr={lr}: {mean_reward:.1f} ± {std_reward:.1f}")

    return resultados

if LUNAR_AVAILABLE:
    resultados_d = estudiar_learning_rate(timesteps=20000)
```

```
=====
VARIANTE D: Learning Rate Sweep
=====
```

Entrenando PPO con lr=0.0001...

lr=0.0001: -708.4 ± 139.8

Entrenando PPO con lr=0.0003...

lr=0.0003: -831.1 ± 540.0

Entrenando PPO con lr=0.001...

lr=0.001: -312.2 ± 76.5

Entrenando PPO con lr=0.003...

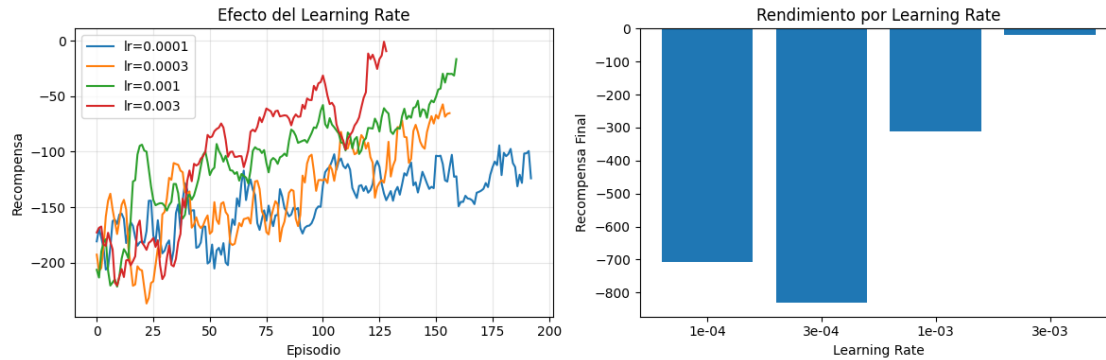
lr=0.003: -18.6 ± 86.4

```
[11]: # Visualizar efecto de learning rate
if LUNAR_AVAILABLE and 'resultados_d' in dir() and resultados_d:
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))

    # Curvas
    for lr, data in resultados_d.items():
        rewards = data['rewards']
        if len(rewards) > 10:
            smoothed = np.convolve(rewards, np.ones(10)/10, mode='valid')
            axes[0].plot(smoothed, label=f'lr={lr}')
    axes[0].set_xlabel('Episodio')
    axes[0].set_ylabel('Recompensa')
    axes[0].set_title('Efecto del Learning Rate')
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

    # Barras
    lrs = list(resultados_d.keys())
    means = [resultados_d[lr]['mean'] for lr in lrs]
    axes[1].bar([f'{lr:.0e}' for lr in lrs], means)
    axes[1].set_xlabel('Learning Rate')
    axes[1].set_ylabel('Recompensa Final')
    axes[1].set_title('Rendimiento por Learning Rate')

plt.tight_layout()
plt.show()
```



9 8. Conclusiones

9.1 ¿Qué aprendimos?

1. **Comparación de algoritmos:**
 - PPO suele ser el más robusto y fácil de usar
 - DQN puede ser más sample-efficient
 - A2C es más simple pero menos estable
2. **Gamma (horizonte temporal):**
 - $\gamma=0.99$ suele ser un buen valor por defecto
 - Valores muy bajos (0.9) no planifican suficiente
 - Valores muy altos (0.999) pueden ser inestables
3. **Learning rate:**
 - $3e-4$ suele funcionar bien para PPO
 - LR muy alto causa inestabilidad
 - LR muy bajo hace el aprendizaje lento
4. **Acciones continuas vs discretas:**
 - SAC funciona bien para acciones continuas
 - Control más fino pero más difícil de aprender

9.2 Referencias

- [PPO Paper](#)
- [DQN Paper](#)
- [SAC Paper](#)
- [Stable-Baselines3 Docs](#)

9.3 Variantes de Entrenamiento — LunarLander

LunarLander es ideal para comparar algoritmos porque existe en dos versiones: - **Discreto** (LunarLander-v3): 4 acciones (no hacer nada, motor izq., motor der., motor principal) - **Continuo** (LunarLanderContinuous-v3): 2 valores reales (potencia motor principal y lateral)

Esto permite comparar algoritmos discretos vs continuos en el mismo entorno.

Variante	Algoritmo	Entorno	Tipo	Nº params
A	PPO	Discreto	On-policy	~50K
B	DQN	Discreto	Off-policy	~50K
C	A2C	Discreto	On-policy	~50K
D	SAC/TD3	Continuo	Off-policy	~100K

9.3.1 Variantes A/B/C — Algoritmos en Entorno Discreto

```
python lunarlander_sb3.py --algorithm PPO      # Var. A
python lunarlander_sb3.py --algorithm DQN      # Var. B
python lunarlander_sb3.py --algorithm A2C      # Var. C
python lunarlander_sb3.py --compare            # Comparar A+B+C
```

Los tres algoritmos resuelven el mismo problema (aterrizar con >200 puntos) pero con filosofías distintas:

PPO (Proximal Policy Optimization) — Variante A - On-policy: aprende solo de experiencias recientes - Clipped surrogate objective: evita actualizaciones demasiado grandes - Robusto y general → algoritmo por defecto en muchos contextos

DQN (Deep Q-Network) — Variante B - Off-policy: replay buffer, aprende de experiencias pasadas - Más eficiente en datos, pero solo funciona con acciones discretas - Puede sobreestimar Q-values (problema de maximización)

A2C (Advantage Actor-Critic) — Variante C - On-policy, más simple que PPO (sin el clip) - Actor: aprende la política | Crítico: aprende el valor del estado - Más rápido por actualización, pero menos estable que PPO

```
[12]: # Variantes A/B/C: comparar PPO, DQN y A2C
# from lunarlander_sb3 import comparar_algoritmos
# resultados = comparar_algoritmos(timesteps=50000)

print("Comparativa A/B/C: PPO vs DQN vs A2C en LunarLander discreto")
print()
print("Criterio de éxito: recompensa media > 200 puntos")
print()
print("Diferencias clave:")
comparativa = """
                PPO          DQN          A2C

Tipo:           On-policy    Off-policy    On-policy
Buffer:         No          Sí (100K)     No
Update freq:    Cada 2048    Cada 4 pasos  Cada 5 pasos
Estabilidad:    Alta         Media         Media-baja
Datos:          Menos effic.  Más effic.    Menos effic.

"""
```

```
print(comparativa)
print("Para comparar visualmente:")
print("  Genera: comparacion_algoritmos.png")
```

Comparativa A/B/C: PPO vs DQN vs A2C en LunarLander discreto

Criterio de éxito: recompensa media > 200 puntos

Diferencias clave:

	PPO	DQN	A2C
Tipo:	On-policy	Off-policy	On-policy
Buffer:	No	Sí (100K)	No
Update freq:	Cada 2048	Cada 4 pasos	Cada 5 pasos
Estabilidad:	Alta	Media	Media-baja
Datos:	Menos efic.	Más efic.	Menos efic.

Para comparar visualmente:

Genera: comparacion_algoritmos.png

9.3.2 Variante D — Entorno Continuo con SAC/TD3

```
python lunarlander_sb3.py --continuous           # SAC (default)
python lunarlander_sb3.py --continuous --algorithm TD3 # TD3
python lunarlander_sb3.py --compare-cont         # Discreto vs Continuo
```

¿Qué cambia en el entorno continuo?

	Discreto	Continuo
Entorno	LunarLander-v3	LunarLanderContinuous-v3
Espacio de acción	Discrete(4)	Box([-1,-1], [1,1])
Acción	índice 0-3	[motor_princ, motor_lat] ²
Control	On/Off	Potencia precisa (0%, 30%, 100%...)
Algoritmos	PPO, DQN, A2C	SAC, TD3

En el discreto: “¿enciendo el motor principal?” → sí/no En el continuo: “¿cuánta potencia al motor principal?” → 0.0 a 1.0

SAC vs TD3 para el entorno continuo **SAC (Soft Actor-Critic)**: - Maximiza recompensa y entropía de la política simultáneamente - $\text{objetivo} = E[\text{reward}] + \gamma \times E[-\log \pi(a|s)]$ - La entropía es adaptativa (se ajusta automáticamente) - Muy explorador → bueno cuando el paisaje de recompensa es complejo

TD3 (Twin Delayed DDPG): - Usa *dos* redes Q (twin) y toma el mínimo → reduce sobreestimación - Actualiza el actor con retraso (delayed, cada 2 pasos del crítico) - Más determinista que SAC, menos exploración - Más estable en entornos con ruido

SAC: maximiza $E[\text{reward}] + \gamma V_t - H(\pi) \leftarrow$ entropía máxima

TD3: minimiza error de los 2 Q-networks y actúa con ruido gaussiano

```
[13]: # Variante D: SAC en entorno continuo
# from lunarlander_sb3 import entrenar_continuo
# model, callback = entrenar_continuo(algoritmo="SAC", timesteps=100000)

print("Variante D: LunarLanderContinuous-v3 con SAC")
print()
print("Espacio de acciones continuo:")
print("  action = [motor_principal, motor_lateral]")
print("  Cada valor  [-1.0, 1.0]")
print("  Negativo = motor apagado")
print()
sac_config = """
SAC(
    "MlpPolicy",
    env,                                # LunarLanderContinuous-v3
    learning_rate=3e-4,
    buffer_size=100000,
    learning_starts=1000,
    batch_size=256,
    gamma=0.99,
    tau=0.005,                          # Soft update del target network
    ent_coef="auto",                    # Entropía adaptativa automática
)
"""
print("Configuración SAC:")
print(sac_config)

print("Para comparar discreto vs continuo:")
print("  from lunarlander_sb3 import comparar_discreto_vs_continuo")
print("  comparar_discreto_vs_continuo(timesteps=50000)")
print("  → Genera: lunarlander_discreto_vs_continuo.png")
```

Variante D: LunarLanderContinuous-v3 con SAC

Espacio de acciones continuo:

action = [motor_principal, motor_lateral]

Cada valor [-1.0, 1.0]

Negativo = motor apagado

Configuración SAC:

```
SAC(
    "MlpPolicy",
    env,                                # LunarLanderContinuous-v3
    learning_rate=3e-4,
```

```

    buffer_size=100000,
    learning_starts=1000,
    batch_size=256,
    gamma=0.99,
    tau=0.005,                # Soft update del target network
    ent_coef="auto",          # Entropía adaptativa automática
)

```

Para comparar discreto vs continuo:

```

from lunarlander_sb3 import comparar_discreto_vs_continuo
comparar_discreto_vs_continuo(timesteps=50000)
→ Genera: lunarlander_discreto_vs_continuo.png

```

9.3.3 Comparativa Final

Aspecto	A: PPO	B: DQN	C: A2C	D: SAC/TD3
Entorno	Discreto	Discreto	Discreto	Continuo
Convergencia	~100K	~80K	~120K	~150K
Estabilidad	Alta	Media	Media	Alta (SAC)
Control	Grosero (4 op.)	Grosero	Grosero	Preciso (continuo)
Curva aprendizaje	Suave	Variable	Ruidosa	Suave

Criterio de éxito: recompensa media > 200 puntos en 10 episodios de evaluación.

Lección: el espacio de acción continuo permite control más preciso pero es más difícil de aprender. SAC y TD3 son los algoritmos estándar para robótica y control continuo.